МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»
Центр образовательных программ топ-уровня в сфере
информационных технологий Образовательного центра Института №8

ЛАБОРАТОРНАЯ РАБОТА

по дисциплине «Операционные системы и архитектура компьютера»

на тему: **«Управление процессами и обмен данными между процессами посредством каналов (pipe)»**

Выполнил:

студент гр. М8О-206БВ-25

Толстопятов Трофим Михайлович

Проверил: Ефимов Д.А.

Оценка:

Дата:

Москва 2026

СОДЕРЖАНИЕ

Цель работы 2
Архитектура решения 3
Описание задачи 3
Схема взаимодействия 3
Схема расстановки файловых дескрипторов 4
Применённые системные примитивы 5
Ключевые фрагменты кода 5
Инициализация IPC и создание дочернего процесса 5
Передача данных и приём результата 6
Обработка ввода и деления на ноль 7
Результаты работы 9
Выводы 12
Приложения 13
Приложение А: ссылка на Git 13

Цель работы

Приобретение практических навыков в:

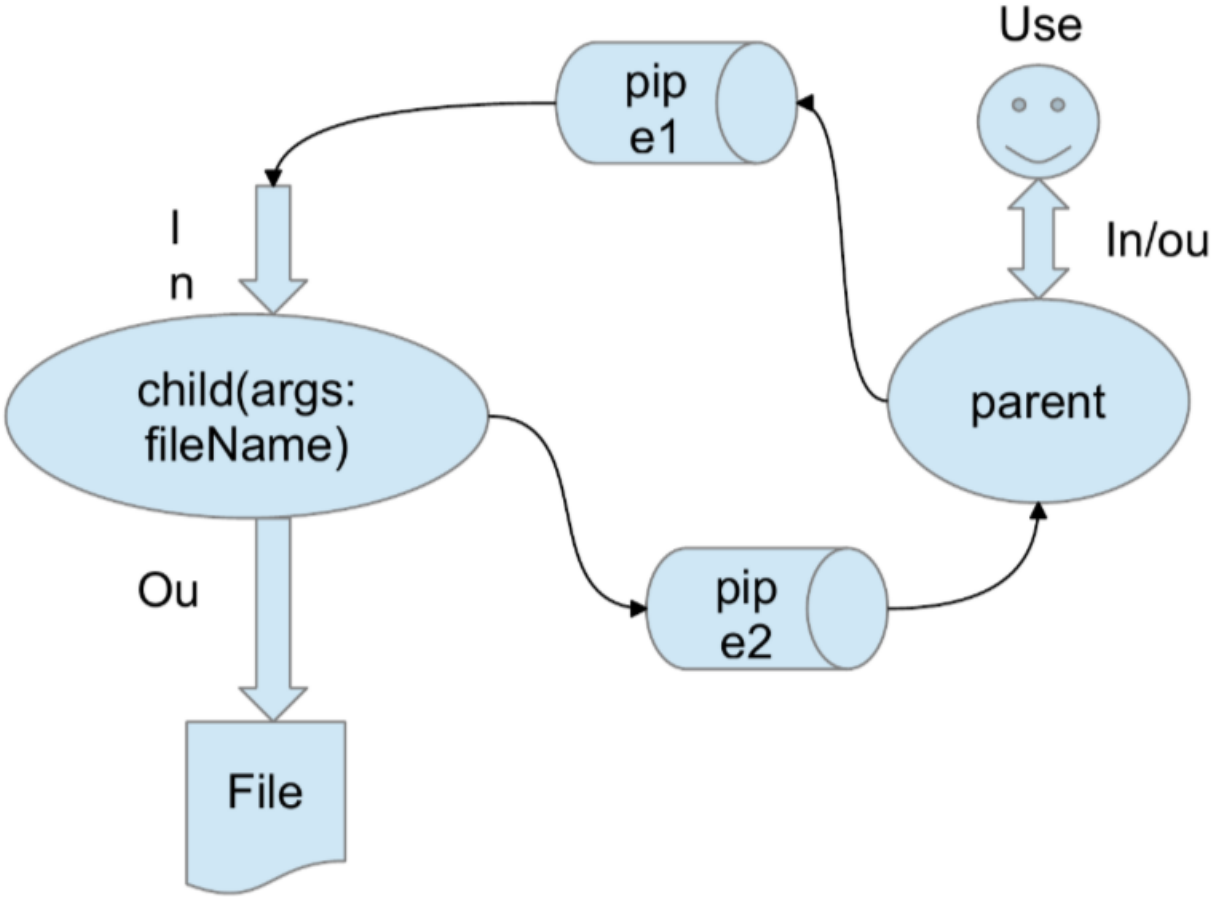
- управлении процессами в ОС Linux средствами POSIX API;
- обеспечении обмена данными между процессами посредством неименованных каналов (pipe);
- обработке системных ошибок, возникающих при работе с процессами и IPC.

Архитектура решения

Описание задачи

Родительский процесс создаёт дочерний. Первой строкой пользователь вводит имя файла, которое передаётся дочернему процессу через argv. Далее пользователь вводит команды вида “число число число <endline>” (числа типа float). Родитель передаёт числа дочернему процессу через pipe1, связанный со стандартным вводом дочернего процесса. Дочерний процесс вычисляет a / b / c для каждой команды и записывает результат в файл. Если встречается деление на 0 — проверка выполняется на стороне дочернего процесса, после чего оба процесса завершаются. При необходимости дочерний процесс передаёт данные родителю через pipe2.

Схема взаимодействия



- parent и child - две разные команды, связь между ними - через аргументы execve (стартовые данные) и pipes.

- pipe1 - однонаправленный канал: parent пишет, child читает через переопределённый stdin.
- pipe2 - однонаправленный канал: child пишет через переопределённый stdout, parent читает.

Схема расстановки файловых дескрипторов

Процесс	fd 0	fd 1	fd 2	Прочее
parent (до fork)	stdin	stdout	stderr	pipe1[0], pipe1[1], pipe2[0], pipe2[1]
parent (после настройки)	stdin	stdout	stderr	pipe1[1] (write), pipe2[0] (read)
child (после настройки)	pipe1[0]	pipe2[1]	stderr	-

Применённые системные примитивы

Примитив	Назначение
fork	Создание дочернего процесса
pipe	Создание неименованного канала
dup2	Переназначение файловых дескрипторов
execve	Замена образа памяти процесса
read/write	Обмен данными через каналы
close	Освобождение файловых дескрипторов
fopen/fclose	Открытие/закрытие файла
waitpid	Ожидание завершения дочернего процесса с последующим сбором статуса
signal	Изменение поведения на сигнал

Ключевые фрагменты кода

Инициализация IPC и создание дочернего процесса

src/parent/main.c:

```
int pipefd1[2], pipefd2[2];
if (pipe(pipefd1) == -1) {
```

```
    perror("pipefd1");
    return 1;
}
if (pipe(pipefd2) == -1) {
    perror("pipefd2");
    return 1;
}

pid_t pid = fork();

if (pid == -1) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    close(pipefd1[1]);
    close(pipefd2[0]);
    if (dup2(pipefd1[0], 0) == -1) {
        perror("dup2 stdin");
        _exit(1);
    }
    if (pipefd1[0] != 0) {
        close(pipefd1[0]);
    }
    if (dup2(pipefd2[1], 1) == -1) {
        perror("dup2 stdout");
        _exit(1);
    }
    if (pipefd2[1] != 1) {
        close(pipefd2[1]);
    }
    char* argv[] = {"./child", fileName, NULL};
    execve("./child", argv, NULL);
    perror("execve");
    fflush(stderr);
    _exit(1);
}

signal(SIGPIPE, SIG_IGN);

close(pipefd1[0]);
close(pipefd2[1]);
```

Основные процессы:

- dup2(pipefd1[0], 0) делает stdin child концом для чтения pipe1
- dup2(pipefd2[1], 1) делает stdout child концом для записи pipe2
- лишние концы труб закрываются для корректной работы
- execve заменяет образ памяти дочернего процесса на child, сохраняя таблицу файловых дескрипторов, содержащую настроенные трубы
- signal заставляет игнорировать SIGPIPE, чтобы write в закрытую трубу не убил родителя сигналом

Передача данных и приём результата

src/parent/main.c:

```
char cin[256];
while (fgets(cin, sizeof(cin), stdin)) {
    if (write(pipefd1[1], cin, strlen(cin)) == -1) {
        if (errno == EPIPE) break;
        perror("write to pipe1");
        break;
    }
}

close(pipefd1[1]);

char buf[256];
ssize_t n;
while ((n = read(pipefd2[0], buf, sizeof(buf))) > 0) {
    fwrite(buf, 1, n, stdout);
}
close(pipefd2[0]);

int status;
if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid");
    return 1;
}
if (WIFEXITED(status) && WEXITSTATUS(status) == 2) {
    return 2;
}
```

Основные процессы:

- write пишет данные в pipe1, пока child не закроет его
- close(pipefd1[1]) - после закрытия конца записи в pipe1 child получает EOF

- чтение из pipe2 идёт до waitpid, чтобы избежать deadlock при заполненном буфере трубы
- статус child анализируется через WIFEXITED/WEXITSTATUS, и при ошибке деления на 0 в child возвращается код 2

Обработка ввода и деления на ноль

src/child/main.c

```
FILE* f = fopen(argv[1], "w");
if (!f) {
    perror("fopen");
    return 1;
}

char cin[256];
float a, b, c;
while (fgets(cin, sizeof(cin), stdin)) {
    if (sscanf(cin, "%f %f %f", &a, &b, &c) != 3) continue;
    if (b == 0.0 || c == 0.0) {
        printf("Child: error of division zero\n");
        fflush(stdout);
        fclose(f);
        return 2;
    }
    a /= b;
    a /= c;
    fprintf(f, "%f\n", a);
}

fclose(f);

return 0;
```

Основные процессы:

- проверка деления на ноль выполняется в дочернем процессе
- сообщение об ошибке деления на ноль пишется в stdout child, который через dup2 связан с pipe2
- При ошибке деления на ноль возвращается код 2

Результаты работы

Базовый сценарий

```
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$ ./parent
../output.txt
10 2 5
10 5 2
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$ cat ../output.txt
1.000000
1.000000
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$
```

Деление на ноль

```
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$ ./parent
../output.txt
10 2 5
1 0 2
Child: error of division zero
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$ echo $?
2
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$
```

Демонстрация конкурентного выполнения

```
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$ ps -ef | grep -E 'parent|child'
tr0f1mka    4923     345  0 00:01 pts/0    00:00:00 ./parent
tr0f1mka    4926    4923  0 00:01 pts/0    00:00:00 ./child ../output.txt
```

- 4923 – ID parent
- 4926 – ID child

Проверка распределения файловых дескрипторов

```
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$ ls -l /proc/4923/fd
total 0
lrwx------ 1 tr0f1mka tr0f1mka 64 Oct  5 00:04 0 -> /dev/pts/0
lrwx------ 1 tr0f1mka tr0f1mka 64 Oct  5 00:04 1 -> /dev/pts/0
lrwx------ 1 tr0f1mka tr0f1mka 64 Oct  5 00:04 2 -> /dev/pts/0
l-wx------ 1 tr0f1mka tr0f1mka 64 Oct  5 00:04 4 -> 'pipe:[26201]'
lr-x------ 1 tr0f1mka tr0f1mka 64 Oct  5 00:04 5 -> 'pipe:[26202]'
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$ ls -l /proc/4926/fd
total 0
lr-x------ 1 tr0f1mka tr0f1mka 64 Oct  5 00:04 0 -> 'pipe:[26201]'
l-wx------ 1 tr0f1mka tr0f1mka 64 Oct  5 00:04 1 -> 'pipe:[26202]'
lrwx------ 1 tr0f1mka tr0f1mka 64 Oct  5 00:04 2 -> /dev/pts/0
l-wx------ 1 tr0f1mka tr0f1mka 64 Oct  5 00:04 3 -> /mnt/f/Vuz/OS/lab2/os_lab2/output.txt
```

Видно, что stdin (0) у child направлен на тот же канал, что и выход 4 у parent. Аналогично stdout (1) у child направлен на канал входа 5 у parent. Лишних дескрипторов нет.

Проверка отсутствия зомби и утечек файловых дескрипторов

```
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$ valgrind --track-fds=yes --leak-check=full ./parent
==5236== Memcheck, a memory error detector
==5236== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==5236== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==5236== Command: ./parent
==5236==
../output.txt
10 2 5
1 0 2

Child: error of division zero
==5236==
==5236== FILE DESCRIPTORS: 3 open (3 inherited) at exit.
==5236==
==5236== HEAP SUMMARY:
==5236==     in use at exit: 0 bytes in 0 blocks
==5236==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==5236==
==5236== All heap blocks were freed -- no leaks are possible
==5236==
==5236== For lists of detected and suppressed errors, rerun with: -s
==5236== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$ ps -eo pid,ppid,stat,cmd | awk '$3 ~ /^Z/'
tr0f1mka@Tr0f1mka:/mnt/f/Vuz/OS/lab2/os_lab2/build$
```

Скриншот подтверждает, что программа не оставляет зомби-процессов, утечек памяти и открытых каналов.

Диагностика через strace

С помощью трассировщика strace была проведена диагностика работы программы

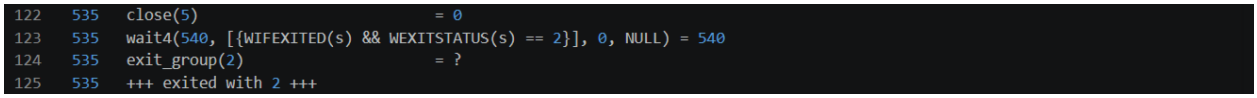
```
34   535   read(0, "../output.txt\n", 1024)  = 14
35   535   pipe2([3, 4], 0)                  = 0
36   535   pipe2([5, 6], 0)                  = 0
37   535   rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
38   535   clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x76e660e56a10) = 540
```

Здесь после чтения имени файла (read(0, "../output.txt\n", 1024)) происходит создание двух неименованных каналов [3, 4] и [5, 6] (pipe2([3, 4], 0) и pipe([5, 6], 0) соответственно). Далее эти каналы будут называться pipe1 и pipe2 соответственно. После идёт создание дочернего процесса с копированием образа памяти родительского процесса (вызов fork).

```
47   540   close(4 <unfinished ...>
48   535   close(3 <unfinished ...>
49   540   <... close resumed>)              = 0
50   535   <... close resumed>)              = 0
51   540   close(5 <unfinished ...>
52   535   close(6 <unfinished ...>
53   540   <... close resumed>)              = 0
54   535   <... close resumed>)              = 0
55   540   dup2(3, 0 <unfinished ...>
56   535   read(0 <unfinished ...>
57   540   <... dup2 resumed>)               = 0
58   540   close(3)                          = 0
59   540   dup2(6, 1)                        = 1
60   540   close(6)                          = 0
61   540   execve("./child", ["./child", "../output.txt"], NULL) = 0
```

После fork идёт настройка каналов: родительский процесс закрывает концы каналов под номерами 3 и 6, а дочерний - 4 и 5, так как они им соответственно не нужны и являются копиями концов каналов, которые используются в противоположном процессе. Также в дочернем процессе происходит переназначение концов 3 и 6 на stdin и stdout соответственно. После вызывается execve, заменяющий образ памяти процесса на нужную нам программу. При этом таблица файловых дескрипторов сохраняется, из-за чего stdin читает из конца 3 канала pipe1, а stdout пишет в конец 6 канала pipe2.

```
93   540   openat(AT_FDCWD, "../output.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
94   540   fstat(0, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
95   540   read(0 <unfinished ...>
96   535   <... read resumed>, "10 2 4\n", 1024) = 7
97   535   write(4, "10 2 4\n", 7)           = 7
98   540   <... read resumed>, "10 2 4\n", 4096) = 7
99   535   read(0 <unfinished ...>
100  540   fstat(3, {st_mode=S_IFREG|0777, st_size=0, ...}) = 0
101  540   read(0 <unfinished ...>
102  535   <... read resumed>, "1 0 2\n", 1024) = 6
103  535   write(4, "1 0 2\n", 6)            = 6
104  540   <... read resumed>, "1 0 2\n", 4096) = 6
105  535   read(0 <unfinished ...>
106  540   fstat(1, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
107  540   write(1, "Child: error of division zero\n", 30) = 30
108  540   write(3, "1.250000\n", 9)          = 9
109  540   close(3)                          = 0
110  540   exit_group(2)                     = ?
111  540   +++ exited with 2 +++
112  535   <... read resumed>, 0x597b8b523010, 1024) = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
113  535   --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=540, si_uid=1000, si_status=2, si_utime=0, si_stime=1 /* 0.01 s */
114  535   read(0, "\n", 1024)               = 1
115  535   write(4, "\n", 1)                 = -1 EPIPE (Broken pipe)
116  535   --- SIGPIPE {si_signo=SIGPIPE, si_code=SI_USER, si_pid=535, si_uid=1000} ---
117  535   close(4)                          = 0
118  535   read(5, "Child: error of division zero\n", 256) = 30
119  535   fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
120  535   write(1, "Child: error of division zero\n", 30) = 30
121  535   read(5, "", 256)                  = 0
```



Далее идёт основная работа программы: дочерний процесс открывает файл на новом файловом дескрипторе, родительский процесс читает консоль и пишет в stdin дочернего процесса через конец 4 канала pipe1. При встрече деления на ноль дочерний процесс отправляет в конец канец канала 5, после данные из буфера записываются в файл и процесс завершается с кодом 2, закрывая концы 4 и 5. Родительский процесс встречает невозможность записи в канал pipe1, что является сигналом завершения дочернего процесса, и закрывает конец 4 канала pipe1. Затем родительский процесс читает буфер pipe2 и выводит его содержимое в stdout, после закрывает конец 5 канала pipe2, после анализирует статус-код завершения дочернего процесса и возвращает соответствующий код работы.

Выводы

В ходе лабораторной работы:

- освоены механизмы создания процессов (fork, execve) и управления ими (waitpid) в POSIX-совместимой ОС.
- реализован двунаправленный обмен данными между процессами через два неименованных канала (pipe): родитель → ребёнок через pipe1, ребёнок → родитель через pipe2.
- изучено перенаправление файловых дескрипторов через dup2: stdin ребёнка связан с pipe1, stdout — с pipe2. Показано, что execve сохраняет таблицу дескрипторов, что позволяет передавать каналы новой программе.
- реализована обработка системных ошибок: проверяются возвраты pipe, fork, dup2, execve, write, waitpid, fopen; ошибки выводятся через perror.
- обработаны краевые случаи:
- деление на 0 — проверяется в дочернем процессе, оба процесса завершаются, родитель возвращает код 2;

- разрыв канала — SIGPIPE игнорируется, write возвращает EPIPE, что корректно обрабатывается;
- отсутствие зомби — родитель всегда вызывает waitpid;
- утечки fd/памяти — подтверждено отсутствие через valgrind и /proc/<pid>/fd.
- программа проверена под strace: показано, что каждый системный вызов, реализующий логику задания, выполняется в нужный момент.

Приложения

Приложение А: ссылка на Git

https://github.com/Tr0f1mka/os_lab2.git